**Name:Samruddhi Sangram Yadav.**

**Roll No.: 560**

## Assignment NO. 3B

## Create four API using Node.JS, ExpressJS and MongoDB for CURD Operations on assignment 2.C.

```
PS E:\wad\assignment3B> npm init -y
Wrote to E:\wad\assignment3B\package.json:
{
  "name": "assignment3b",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}
PS E:\wad\assignment3B> npm install express mongoose body-parser
added 82 packages, and audited 83 packages in 6s
23 packages are looking for funding
  run `npm fund` for details
found 0 vulnerabilities
PS E:\wad\assignment3B> npm install cors
added 2 packages, and audited 85 packages in 1s
24 packages are looking for funding
```

run `npm fund` for details

found 0 vulnerabilities

### server.js

```
const express = require("express");
const mongoose = require("mongoose");
const bodyParser = require("body-parser");
const cors = require("cors");
const app = express();
app.use(cors());
app.use(bodyParser.json());
mongoose.connect("mongodb://127.0.0.1:27017/crudDB")
.then(() => console.log("MongoDB Connected"))
.catch(err => console.log(err));
const UserSchema = new mongoose.Schema({
  name: String,
  email: String,
  age: Number
});
const User = mongoose.model("User", UserSchema);
// CREATE
app.post("/users", async (req, res) => {
  try {
    const user = new User(req.body);
    const savedUser = await user.save();
    res.status(201).json(savedUser);
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
```

```
});
// READ
app.get("/users", async (req, res) => {
  try {
    const users = await User.find();
    res.json(users);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
// UPDATE
app.put("/users/:id", async (req, res) => {
  try {
    const updatedUser = await User.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true }
    );
    res.json(updatedUser);
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
});
// DELETE
app.delete("/users/:id", async (req, res) => {
  try {
    await User.findByIdAndDelete(req.params.id);
    res.json({ message: "User deleted successfully" });
```

```
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

## Index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>CRUD Application</title>
  <style>
    body {
      font-family: Arial;
      margin: 30px;
    }
    input {
      padding: 8px;
      margin: 5px;
    }
    button {
      padding: 8px;
      margin: 5px;
      cursor: pointer;
    }
    table {
```

```
        border-collapse: collapse;
        width: 100%;
        margin-top: 20px;
      }
      table, th, td {
        border: 1px solid black;
      }
      th, td {
        padding: 10px;
        text-align: center;
      }
    </style>
</head>
<body>
    <h1>CRUD Application</h1>
    <!-- FORM -->
    <input type="hidden" id="userId">
    <input type="text" id="name" placeholder="Enter Name">
    <input type="email" id="email" placeholder="Enter Email">
    <input type="number" id="age" placeholder="Enter Age">
    <button onclick="saveUser()">Save User</button>
    <!-- TABLE -->
    <table>
      <thead>
        <tr>
          <th>Name</th>
          <th>Email</th>
          <th>Age</th>
```

```
            <th>Actions</th>
          </tr>
        </thead>
        <tbody id="userTableBody">
        </tbody>
    </table>
<script>
const apiURL = "http://localhost:3000/users";
// LOAD USERS WHEN PAGE OPENS
window.onload = fetchUsers;
// FETCH USERS
async function fetchUsers() {
    const response = await fetch(apiURL);
    const users = await response.json();
    let tableData = "";
    users.forEach(user => {
        tableData += `
            <tr>
                <td>${user.name}</td>
                <td>${user.email}</td>
                <td>${user.age}</td>
                <td>
                    <button onclick="editUser(
                        '${user._id}',
                        '${user.name}',
                        '${user.email}',
                        '${user.age}'
                    )">
```

```
                Edit
              </button>
              <button onclick="deleteUser('${user._id}')"> Delete </button>
            </td>
    </tr>
      `;
  });
  document.getElementById("userTableBody").innerHTML = tableData;
}
// SAVE USER (CREATE + UPDATE)
async function saveUser() {
  const id = document.getElementById("userId").value;
  const user = {
    name: document.getElementById("name").value,
    email: document.getElementById("email").value,
    age: document.getElementById("age").value
  };
  // UPDATE USER
  if(id) {
    await fetch(` ${apiURL}/${id}` , {
      method: "PUT",
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify(user)
    });
  }
  // CREATE USER
```

```
    else {
      await fetch(apiURL, {
        method: "POST",
        headers: {
          "Content-Type": "application/json"
        },
        body: JSON.stringify(user)
      });
    }
    clearForm();
    fetchUsers();
}
// EDIT USER
function editUser(id, name, email, age) {
    document.getElementById("userId").value = id;
    document.getElementById("name").value = name;
    document.getElementById("email").value = email;
    document.getElementById("age").value = age;
}
// DELETE USER
async function deleteUser(id) {
    await fetch(` ${apiURL}/${id}` , {
      method: "DELETE"
    });
    fetchUsers();
}
// CLEAR FORM
function clearForm() {
```

```
    document.getElementById("userId").value = "";
    document.getElementById("name").value = "";
    document.getElementById("email").value = "";
    document.getElementById("age").value = "";
}
</script>
</body>
</html>
```

**********OUTPUT**********

```
PS E:\wad\assignment3B> node server.js
Server running on port 3000
MongoDB Connected
```


```
PS E:\wad\assignment3B> mongosh
Current Mongosh Log ID: 6a01bde0121cfd6ce5cebea3
Connecting to:
mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.5.8
```

Using MongoDB: 8.2.0

Using Mongosh: 2.5.8

mongosh 2.5.10 is available for download:
https://www.mongodb.com/try/download/shell

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

---

The server generated these startup warnings when booting

2026-04-27T17:21:41.257+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted

---

```
test> use crudDB
switched to db crudDB
crudDB> show collections
users
crudDB> db.users.find()
crudDB> db.users.find()
[
 {
  _id: ObjectId('6a01c1db087a07a5ea270cce'),
  name: 'SAMRUDDHI YADAV',
  email: 'samruddhi92092@gmail.com',
  age: 21,
  __v: 0
 }
]
crudDB>
```